AI 產品藍圖議案

組別：小組名稱

組員：呂紹銘(D1133324)、張仁維(D1149932)、吳宸宇(D1173579)、林富閎(D1185501)

一、 產品概述

1. 情境名稱：逢甲學生的美食避雷針與專屬推薦指南
2. 服務對象：逢甲大學學生（尤其是有選擇障礙的同學、剛入學的大一新生）、教職員，以及經常在逢甲商圈覓食的族群。
3. 一句話目標：透過分析使用者的當下需求與真實評價，精準分類並推薦最適合的逢甲美食，同時自動過濾掉品質不穩定的「雷店」。
4. 成功指標：
   (1) 減少猶豫時間：有效縮短學生每天決定午/晚餐的時間（例如：從平均猶豫 10 分鐘降低至 3 分鐘內決定）。
   (2) 降低踩雷率：使用者依照推薦前往用餐後，給予負評或回報「踩雷」的比例顯著下降。
   (3) 提升採納率：系統推薦的餐廳名單，使用者實際點擊查看路線並前往用餐的比例提升。

二、 多任務對照表

任務類型	輸入	輸出	方案 A vs 方案 B（各 1-2 句，需寫取捨）
分類(食物品項意圖分類)	1. 使用者輸入文字(字串)2. 當下時間(時間數值)3. 學生所在經緯度 (數值向量)4. 預算上限 (數值)5. 當下天氣狀況(類別)	類別(如：便當、拉麵、小吃、火鍋等)	方案 A (TF-IDF + 隨機森林)： 運算速度快且建置成本低，但無法理解口語化的上下文語意（例如：「想吃熱呼呼的湯」可能分錯）。 方案 B (BERT 語意模型)： 能精準理解學生複雜的自然語言需求，但需要較高的運算資源與延遲時間。取捨： 為了讓學生有更好的搜尋體驗，選擇方案 B，即使運算成本稍高。
預測數值 (迴歸)(預測餐廳排隊等待時間)	1. 餐廳 ID (類別)2. 星期幾 (類別)3. 當下時間點 (時間數值)4. 歷史平均等待時間 (數值)5. 該時	數值(預估需等待的「分鐘數」)	方案 A (線性迴歸 Linear Regression)： 模型簡單、具高度可解釋性，但無法處理複雜的非線性關係（如突發大雨導致學餐爆滿）。 方案 B (XGBoost)： 能精準捕捉多變數間的非線性關係，預測準確率高，但模型像黑盒子較難解

	段逢甲商圈人流預估(數值)		釋原因。取捨： 對趕早八或下午有課的學生來說「準確度」最重要，故選擇方案 B。
推薦(個人化防雷美食推薦)	1. 學生 User ID (類別)2. 學生歷史評分紀錄 (數值陣列)3. 餐廳 ID (類別)4. 餐廳綜合防雷分數 (數值)5. 餐廳與學生的距離 (數值)	清單(Top 5 最適合該學生的餐廳名單)	方案 A (內容過濾 Content-based)： 能輕易推薦新開的店家（解決冷啟動），但推薦結果容易過於單一（吃過拉麵就一直推拉麵）。方案 B (協同過濾 Collaborative)： 能根據「口味相似的同學」給出充滿驚喜的推薦，但對剛辦帳號的大一新生無法給出好建議。取捨： 為了兼顧多樣性與防雷，實務上可採混合兩者的方案，或初期先用方案 A 累積數據。
異常偵測(自動揪出雷店或假評價)	1. 餐廳 ID (類別)2. 單日新增評論數 (數值)3. 評論關鍵字情緒分數 (數值)4. 1 星負評佔比 (數值)5. 短期評分波動率 (數值) 6. 評論者帳號權重/信任度	類別(布林值：正常 / 異常警告)	方案 A (統計規則 Z-score)： 設定簡單、邏輯明確，但只能抓到極端值，容易被店家用「買機器人刷好評」的手法騙過。方案 B (孤立森林 Isolation Forest)： 能在多維度資料中找出隱蔽的異常模式（如大量假好評掩蓋真實負評），但需要調校參數且需要較多歷史數據。取捨： 為了落實真正的「防雷」，選擇方案 B 才能有效揪出品質不穩或造假的店家。

三、 架構流程圖

[階段一：資料來源 Data Sources]

├── 外部開放資料 (Google 評價、Dcard 留言)

├── 系統內部資料 (餐廳建檔、學生歷史輪廓)

└── 即時動態資料 (當下搜尋文字、GPS、天氣)

│

▼ (資料流入)

[階段二：清理與轉換 Data Transformation]

├── 數據清理 (填補空值、過濾無效廣告)

├── 資料去識別化與模糊化（Anonymization & Obfuscation）

├── 文字轉向量 (BERT 轉換口語搜尋與評論)

├── 編碼轉換 (類別資料 One-Hot Encoding)

└── 數值正規化 (距離、預算等數值縮放)

│

▼ (特徵工程完成，進入模型)

[階段三：模型與規則 Models & Rules]

├── 分類模型：BERT 語意模型 ──────> (判斷想吃的品項)

├── 迴歸模型：XGBoost ──────────> (預測需等候時間)

├── 異常偵測：孤立森林 ──────────> (揪出洗評價雷店)

└── 推薦系統：混合協同與內容過濾 ──> (計算個人化配對)

│

▼ (模型運算完畢，產出結果)

[階段四：輸出應用 Output Applications]

├── 分流結果：自動切換至對應的食物分類介面

├── 預測顯示：介面提示「預估等候：15 分鐘」

├── 警示標籤：餐廳圖示旁顯示「近期評價異常警告」

└── 推薦清單：首頁生成「Top 5 專屬防雷美食名單」

四、 AI 協作紀錄

1. 紀錄一：

(1) 我問 AI：請幫我發想逢甲美食防雷系統中，關於「分類」任務的輸入欄位，至少要五個。

(2) AI 回答摘要：AI 建議了使用者輸入文字、當下時間、經緯度、預算上限和當下天氣狀況等五個特徵。

(3) 我修正/補充了：我把「當下天氣狀況」改成「有無交通工具」，因為逢甲商圈周邊很難停車，走路或騎車的差異會大幅影響餐廳選擇。

2. 紀錄二：

(1) 我問 AI：在異常偵測任務中，要揪出假評論或雷店，方案 A 統計規則（Z-score）跟方案 B 孤立森林哪個比較好？

(2) AI 回答摘要：AI 說明 Z-score 只能抓極端值容易被假帳號破解，孤立森林能找出多維度異常模式但需要較多歷史數據。

(3) 我修正/補充了：我把「直接採用孤立森林」改成「初期先用 Z-score 搭配關鍵字過濾，後期再導入孤立森林」，因為新系統剛上線時評價數據可能還不夠多。

3. 紀錄三：

(1) 我問 AI：我們的目標是逢甲美食避雷，產品概述中的成功指標設定為「提升媒合率」這樣適合嗎？

(2) AI 回答摘要：AI 認為可以，並建議可以具體將其量化為使用者查看推薦後，實際前往餐廳用餐的比例。

(3) 我修正/補充了：我把「提升媒合率」改成「降低學生用餐後給予負評的比例」，

因為我們這組的核心主軸是「防雷」，直接看負評下降率更能反映產品是否成功。

四、 Red Team 修正紀錄

- **漏洞 1：隱私資料外洩與過度追蹤風險**
  - **修正前：** 系統直接接收並儲存學生的「所在經緯度」與「學生歷史評分紀錄」進行模型運算。若資料庫防護不足，恐導致學生的真實行蹤軌跡與個人偏好遭到惡意存取或外洩。
  - **修正後：** 在「階段二：清理與轉換 Data Transformation」 中，新增「資料去識別化與模糊化（Anonymization & Obfuscation）」流程。例如：將精確的 GPS 座標轉換為較大範圍的商圈區塊網格（Grid），並對 User ID 進行雜湊（Hash）處理，確保推薦系統在不侵犯隱私的狀態下仍能維持個人化推薦。
- **漏洞 2：惡意水軍的「反向」資料毒化攻擊（Data Poisoning）**
  - **修正前：** 異常偵測模型（孤立森林）主要依賴「單日新增評論數」、「1 星負評佔比」等數值 來揪出雷店或假評價。這存在一個漏洞：同業競爭者可利用大量免洗假帳號，對優質店家惡意狂刷 1 星負評，藉此觸發系統的「異常警告」標籤，導致好店被誤判為雷店。
  - **修正後：** 在異常偵測的輸入欄位中，加入「評論者帳號權重/信任度」特徵。例如：有綁定學校信箱認證的帳號（或是歷史互動良好的老用戶）給出的評分權重較高；而剛註冊、無頭像的匿名帳號若給出極端評分，則大幅降低其權重。以此提升模型對惡意負評攻擊的抵抗力。
- **漏洞 3：突發性黑天鵝事件導致的預測失準**
  - **描述：** 當發生無歷史數據可參考的極端突發事件（例如：商圈突發大停電、或是某知名網紅無預警現身某餐廳引發排隊暴動）時，XGBoost 迴歸模型 所預測的「等待時間」會與現實產生極大誤差。
  - **（可不修，理由如下）：** 突發性黑天鵝事件缺乏足夠的訓練資料（Training data）供模型學習，且發生機率極低。若要為了這類極端特例建立「即時社群突發事件爬蟲與警報系統」，將大幅增加建置成本與系統延遲。這違背了本產品為學生「有效縮短決定午/晚餐的時間（降低至 3 分鐘內）」 的核心目標。實務工程上，在資源有限的情況下應妥協於泛用性，僅需在前端介面加註「預測時間僅供參考，請依現場實際狀況為主」的免責提示即可，不需為此大幅改動後端模型架構